

Weight Initialization in Neural Networks

Task 0 | Principles, equations and practical selection

Weight initialization determines the scale and diversity of a network before learning. A suitable scheme breaks hidden-unit symmetry and keeps activations and gradients at workable magnitudes, supporting stable training and faster convergence. The appropriate scale depends on layer dimensions and activation functions; initialization does not guarantee a global optimum.

Why the starting scale matters

$$z^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}, \quad h^{(l)} = f(z^{(l)})$$

Backpropagation multiplies layer Jacobians. Repeated contraction can produce vanishing gradients; repeated amplification can produce exploding gradients. Large preactivations can also saturate sigmoid or tanh, making their derivatives small. A small update is therefore not proof of a local minimum: inspect gradient norms and the learning rate.

$$\text{Var}(z) \approx n_{in} \text{Var}(w) \text{Var}(x)$$

This approximation assumes independent inputs and weights, zero-mean weights, and negligible cross-covariances. Write n_{in} for fan-in and n_{out} for fan-out. Here $N(0,v)$ uses **variance** v , whereas the standard deviation is $\sqrt{v}$; $U(-a,a)$ has variance $a^2/3$.

Zero initialization

$$W = 0, \quad b = 0$$

Identical hidden units have identical outputs and, under symmetric connections, identical updates. Initializing every weight to zero can also block gradients into earlier layers. Zero weights are inappropriate for ordinary multilayer hidden networks with any activation. Zero *biases* are often valid when weights break symmetry; zero initialization can also work in convex single-layer models such as logistic regression. Its simplicity is outweighed by the hidden-network limitation.

Random initialization with a fixed scale

$$w \sim \mathcal{N}(0, \sigma^2) \quad \text{or} \quad w \sim U(-a, a)$$

Independent random values break symmetry. However, a fixed σ ignores fan-in, fan-out and activation behavior. An overly small scale may attenuate signals across layers; an overly large scale can amplify them or saturate bounded activations. This is a useful baseline or a deliberately calibrated special case, but it is not an activation-specific default.

Xavier or Glorot initialization

$$w \sim \mathcal{N}\left(0, \frac{2g^2}{n_{in} + n_{out}}\right), \quad a = g\sqrt{\frac{6}{n_{in} + n_{out}}}$$

Sample from the normal distribution above or $U(-a,a)$. The basic derivation uses gain $g=1$ and balances approximate forward and backward scales. It is commonly associated with linear or tanh-like layers; activation-specific gains may be used. Sigmoid saturation and nonzero activation means remain concerns, and unit-gain Xavier does not compensate for ReLU gating.

He or Kaiming initialization

$$w \sim \mathcal{N}(0, \frac{2}{n_{in}}), \quad w \sim \mathcal{U}(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}})$$

For a symmetric preactivation distribution, ReLU halves the second moment: $E[\text{ReLU}(z)^2] = E[z^2]/2$. He initialization compensates with variance $2/n_{in}$. This is a second-moment argument; the output variance is not exactly halved because ReLU changes the mean. It is a strong starting choice for ReLU hidden layers, but does not eliminate dead neurons or optimization problems.

$$\text{Var}(w) = \frac{2}{(1 + a^2)n_{in}} \quad \text{for a leaky slope } a$$

Use the adjusted gain for Leaky ReLU or an initial PReLU slope. Fan-in mode targets forward propagation; fan-out mode targets backward scale. Learned slopes can change after initialization, so the starting calculation is approximate.

LeCun initialization

$$w \sim \mathcal{N}(0, \frac{1}{n_{in}}), \quad w \sim \mathcal{U}(-\sqrt{\frac{3}{n_{in}}}, \sqrt{\frac{3}{n_{in}}})$$

Fan-in scaling preserves approximate preactivation variance under the stated assumptions. LeCun normal is especially relevant to SELU self-normalizing networks, together with standardized inputs, suitable feedforward structure and AlphaDropout if dropout is needed. This conditional self-normalization is not guaranteed for arbitrary architectures. The basic scheme is also suitable for approximately linear responses, but does not include ReLU's factor of two.

Worked numerical example

For $n_{in}=100$ and $n_{out}=200$, the unit-gain scales are:

Method	Weight variance	Normal standard deviation	Uniform bound a
Xavier	$2/300 = 0.00667$	0.08165	0.14142
He for ReLU	$2/100 = 0.02000$	0.14142	0.24495
LeCun	$1/100 = 0.01000$	0.10000	0.17321

A reproducible illustration

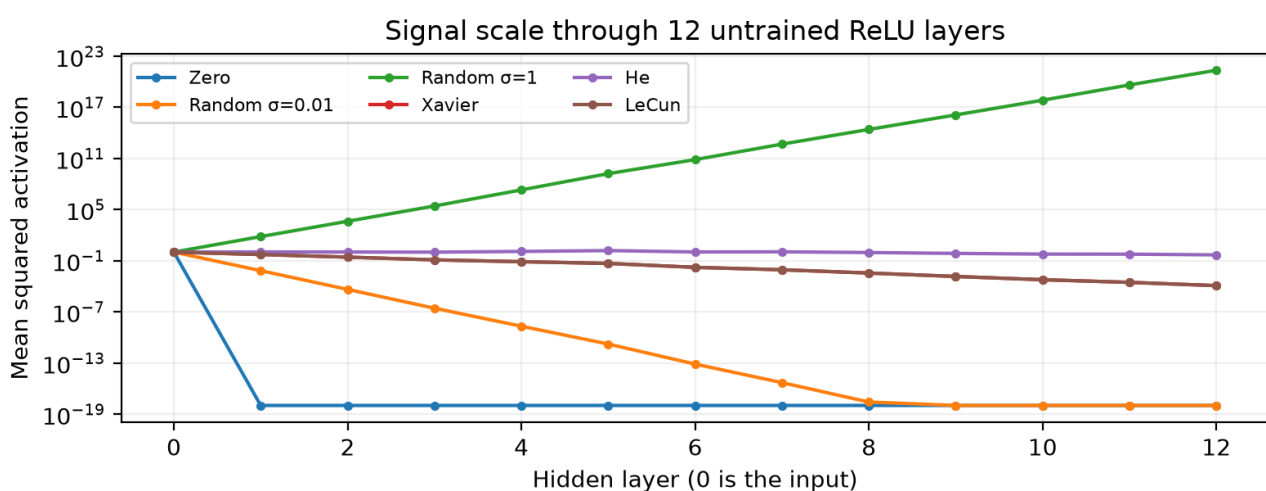


Figure 1. Measured mean squared activations in 12 untrained width-128 ReLU layers, 256 standard-normal inputs, seed 42, zero biases. Values below 10^{-18} are floored for the log plot. Xavier and LeCun coincide for equal fan-in and fan-out with gain 1. This demonstrates initial signal scaling, not training accuracy or a measured convergence-speed ranking. The script and numeric data accompany this report.

Comparison and selection

Method	Suitable starting use	Main advantage	Main limitation
Zero weights	Convex single-layer cases; zero biases with random weights	Simple and deterministic	Hidden-unit symmetry and blocked gradients
Fixed-scale random	Controlled baseline or specialized design	Breaks symmetry	Scale ignores width and activation
Xavier Glorot	Linear and tanh-like layers with appropriate gain	Balances forward and backward scales	Assumptions and saturation; unit gain is weak for deep ReLU
He Kaiming	ReLU; adjusted gain for leaky variants	Accounts for rectifier gating	Does not prevent dead units or later instability
LeCun	SELU with self-normalizing conditions; linear-like responses	Preserves approximate forward variance	No backward balance or ReLU compensation by itself

Practical implications for training

Stability. Match the initialization to the activation and layer shape, normalize inputs where appropriate, and monitor activation statistics, gradient norms and finite losses. Very large gradients may require learning-rate changes and gradient clipping; these address different mechanisms from choosing a suitable initial scale.

Convergence. A useful scale can avoid long saturation plateaus and preserve informative gradients, allowing learning to begin efficiently. Faster convergence in a particular architecture is an empirical claim, not a universal ranking of initialization methods. Optimizer, depth, normalization and data scaling still matter.

Architecture. Recurrent networks require gate-aware handling. LSTM sigmoid/tanh gates should not all receive a ReLU-specific initialization merely because a different layer uses ReLU. Common starting practices include Xavier input matrices and orthogonal recurrent matrices; output-layer initialization follows its own role.

Selection examples

For a ReLU multilayer perceptron, start with He and typically zero biases. For a tanh network, use Xavier with an appropriate gain and inspect saturation. For a SELU feedforward network, use LeCun normal and preserve the self-normalizing assumptions. Keep the random seed fixed when comparing schemes, and repeat across seeds before drawing strong performance conclusions.